DNS Traffic Management: GeoDNS, Weighted, Failover & Anycast

send each user to the right region, and where DNS falls short

Watch the 4-min video

In one line: an authoritative DNS server can give different answers to different users (by location, measured latency, weights or health), which routes traffic between regions; but failover through DNS is slow because of health-check time and caching, so use an anycast global load balancer when you need seconds.

1 One name, different answers

Your app runs in two regions: **eu-west** in Ireland and **us-east** in Virginia. Users in Paris should land in Europe, users in New York in the US, and if Europe goes down, everyone should end up in us-east. DNS can do all of this.

The trick is that an authoritative DNS server doesn't have to give everyone the same answer. With **GeoDNS** (geolocation routing), it looks at where the query comes from and returns the IP of the closest region. A query from Paris gets the eu-west load balancer; a query from New York gets us-east.

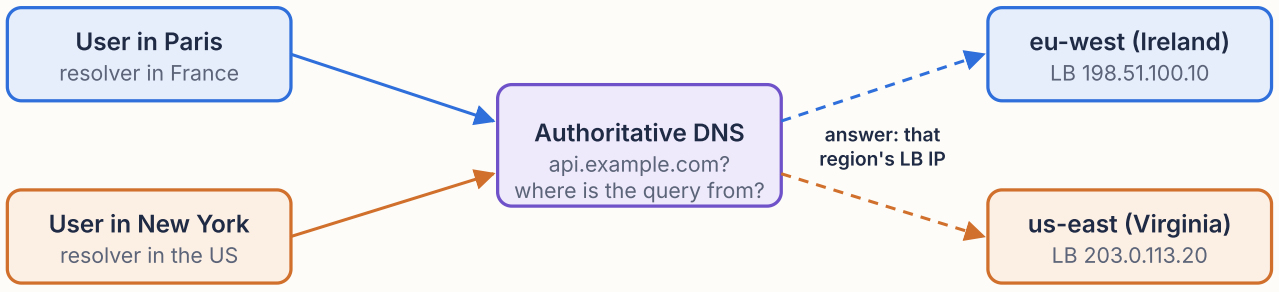


Figure 1. GeoDNS: the same question gets a different answer depending on where it comes from.

Latency-based routing goes one step further. Instead of guessing from geography, the DNS provider measures network latency from different networks to each region and returns the region that is actually fastest for that network. That matters where geography misleads: a user in one country may have a faster path to a region further away. **AWS Route 53** offers both, as well as the policies below.

2 Weighted routing

Weighted routing splits traffic by percentage. Give the current version weight 90 and a new one weight 10, and about 10% of answers point at the new one. It is handy for **canary releases**, **gradual migrations** (move a region or a provider in steps) and **balancing load** between regions of different size.

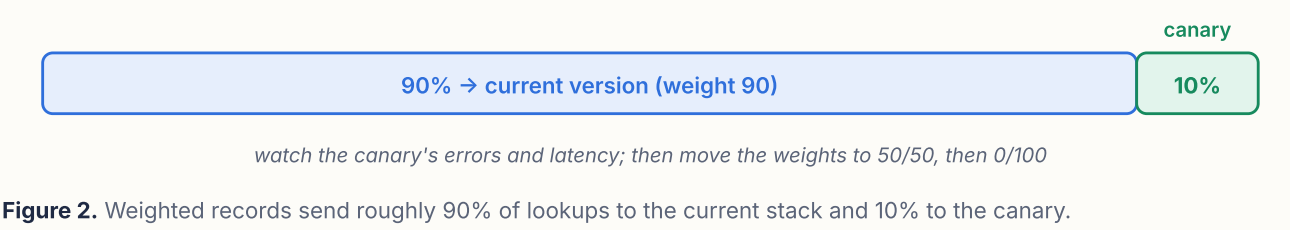


Figure 2. Weighted records send roughly 90% of lookups to the current stack and 10% to the canary.

The split is approximate. It applies to *DNS answers*, not to requests: one big resolver can serve thousands of users the same cached answer for a whole TTL. Over many resolvers it averages out, but don't expect an exact 10%.

3 Failover with health checks

The DNS provider runs **health checks** against each region every few seconds (for example an HTTP request to `/health` every 10 or 30 seconds, from several locations). If eu-west fails several checks in a row, it is marked **unhealthy**, and DNS starts answering with us-east instead. When eu-west recovers and passes its checks again, traffic moves back.

Policies combine: geolocation or latency routing picks the preferred region, and health checks remove a region from the answers when it fails.

4 Anycast: one IP in many places

Anycast is a different tool. The **same IP address** is announced from many locations around the world, and internet routing (BGP) delivers each packet to the nearest one in network terms. Nothing about the DNS answer changes: everyone gets the same IP.

DNS providers use anycast for their own name servers. That makes lookups fast (you reach a nearby server) and spreads out attacks: a DDoS flood is split across every location instead of hitting one. Global load balancers and CDNs use it too, which is the fix for slow DNS failover.

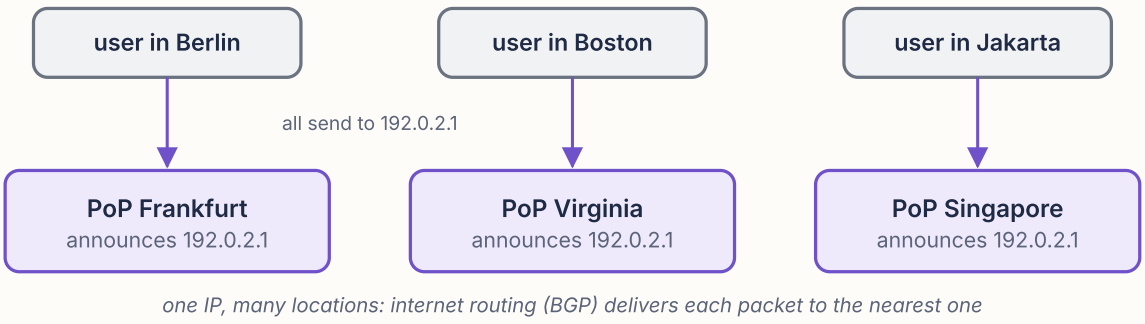
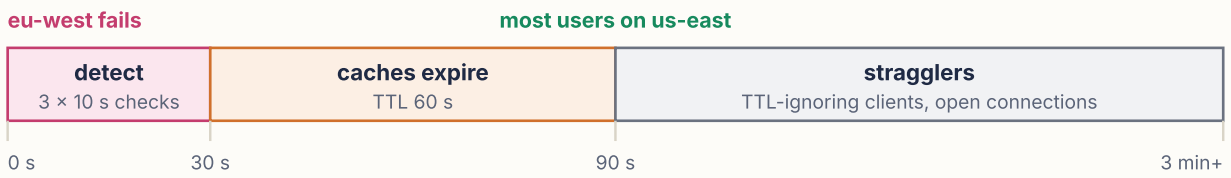


Figure 3. Anycast: three points of presence announce 192.0.2.1. Each user's packets reach the closest one; if one location fails, routing sends them to the next.

5 Why DNS failover is slow

- **Detection.** Several failed health checks are needed to avoid flapping: 3 checks at 10 s is 30 s; at 30 s it is a minute and a half.
- **Caching.** Resolvers keep the old answer until its TTL expires, and some ignore TTLs completely.
- **Connections.** Apps keep their existing connections open (keep-alive, connection pools, WebSockets) and won't look up DNS again until they reconnect.
- **Wrong location.** GeoDNS sees the *resolver's* location, not the user's. A user in Paris whose resolver is far away may land in the wrong region. (Some resolvers forward part of the client's subnet, called EDNS Client Subnet, which helps.)



a global load balancer with an anycast IP shifts traffic in seconds: no DNS answer has to change

Figure 4. A rough DNS failover timeline with fast checks and a 60 s TTL: about a minute and a half for most users, minutes or more for the rest.

WORKED EXAMPLE: THE FAILOVER BUDGET

Health checks every **10 s**, failure after **3** misses: detection takes about **30 s**. The record's TTL is **60 s**, so caches may serve the dead region for up to another minute: about **90 s** total for most users. With 30 s checks and a 300 s TTL it becomes 90 + 300 ≈ **6.5 minutes**. If the target is "under 10 seconds", DNS can't meet it. Put a **global load balancer with an anycast IP** in front (for example AWS Global Accelerator or Cloudflare): its health checks move traffic between regions behind the same IP, so no cache has to expire.

COMMON MISTAKES

- Promising near-instant failover with DNS alone.
- A long TTL on a failover record. The failover can only be as fast as the TTL allows.
- Assuming GeoDNS knows where the user is. It knows where the resolver is.
- Expecting weights to split traffic exactly.
- Health checks that only test "is the web server up", not whether the region can actually serve requests (database, dependencies).

6 Key terms

GeoDNS: answer by the query's location

Health check: probe that marks a target healthy or not

BGP: internet routing between networks

Latency-based routing: answer with the fastest measured region

DNS failover: stop answering with an unhealthy region

Canary: small share of traffic on a new version

Weighted routing: split answers by percentage

Anycast: one IP announced from many places

Global load balancer: anycast front door across regions

7 Interview questions

Q1. How can DNS route users to the nearest region? BASIC

GeoDNS answers based on where the query comes from; latency-based routing returns the region with the lowest measured latency from the user's network. Both are features of the authoritative DNS provider, such as Route 53.

Q2. What is anycast? BASIC

The same IP address announced from many locations. Internet routing delivers each packet to the nearest one. DNS providers use it for their name servers: lookups are fast and DDoS traffic is spread across every site.

Q3. Why is DNS failover slow? BASIC

Health checks need time to detect the failure (often 30–90 s), then cached answers live until their TTL expires, sometimes longer when clients ignore TTLs, and open connections stay on the old region.

Q4. Roll out a new version to 5% of users at the DNS level. How? SCENARIO

Weighted records: 95 for the current stack, 5 for the new one. Watch error rate and latency, then step up to 25, 50, 100. Keep the TTL short (60 s) so you can roll back quickly, and remember the split is approximate.

Q5. Users in Paris sometimes land in us-east. Why? SCENARIO

GeoDNS sees the resolver's location, not the user's. If they use a resolver located in the US, or one without EDNS Client Subnet, they get the US answer. Latency-based routing or an anycast front door reduces this.

Q6. The requirement is failover between regions in under 10 seconds. Design it. SCENARIO

Don't rely on DNS. Put a global load balancer with an anycast IP in front of both regions; it health-checks the regional backends and shifts traffic within seconds behind the same IP. Keep DNS as a slower backup layer.

Q7. You set TTL to 60 s, but after failover some traffic hits the dead region for 30 minutes. Why? TRICKY

Some resolvers and client runtimes cache longer than the TTL, and long-lived connections (pools, WebSockets, HTTP keep-alive) never re-resolve until they break. Close idle connections periodically and have clients retry against a fresh lookup.

Q8. Weighted 90/10, but the canary gets 30% of traffic. How? TRICKY

Weights apply to DNS answers, not requests. A few large resolvers (a big ISP, a corporate network) that happened to get the canary answer serve it to all their users for the whole TTL. It evens out over time, not per minute.

Next: SD-N-018 · HTTP part 1: requests, responses and methods backendroadmap.com